Árvores

PROF. RICARDO MESQUITA

Introdução

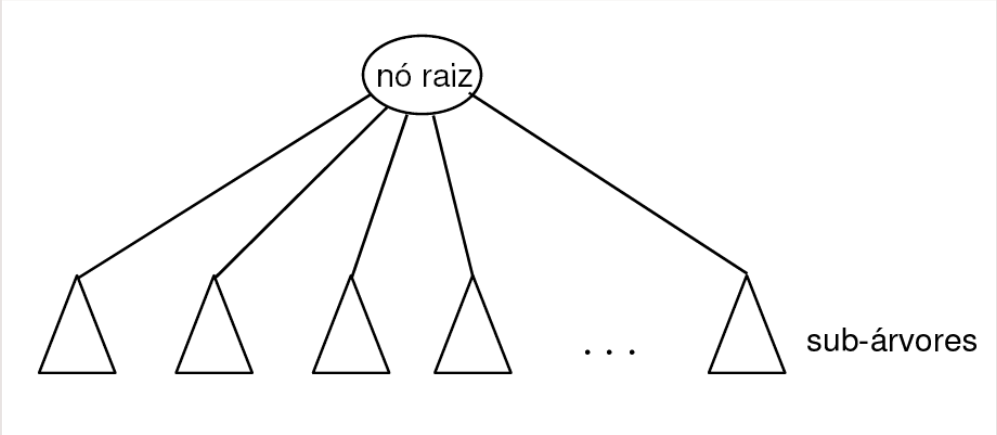
- As estruturas estudadas até agora não são adequadas para representar dados que devem ser dispostos de *maneira hierárquica*.

- Exemplo:
  - Hierarquia de diretórios
  - Árvore genealógica

- Árvores são estruturas de dados adequadas para representação de hierarquias

Definição Recursiva de Árvore

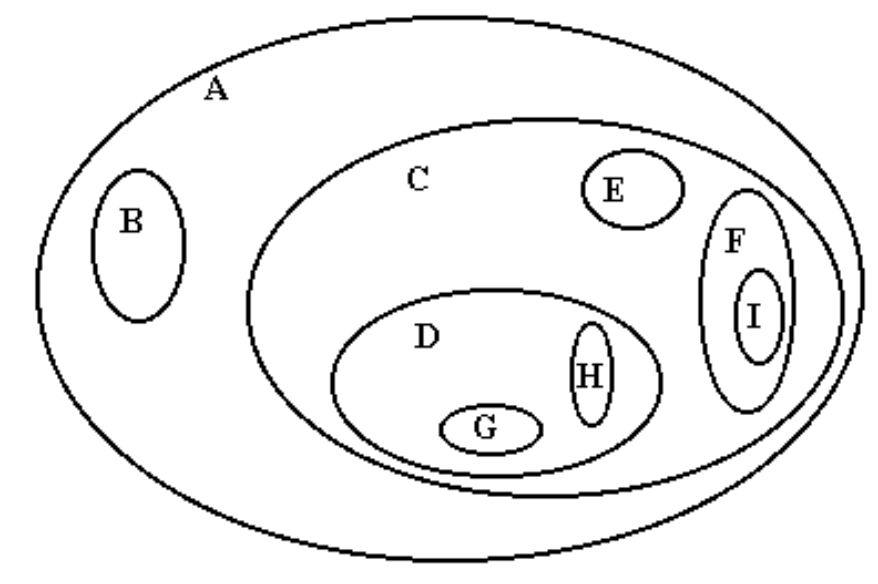
- Uma árvore é um conjunto de nós tal que:
  - existe um nó r, denominado raiz, com zero ou mais subárvores, cujas raízes estão ligadas a r
  - os nós raízes destas subárvores são os filhos de r
  - os nós internos da árvore são os nós com filhos
  - as folhas ou nós externos da árvore são os nós sem filhos



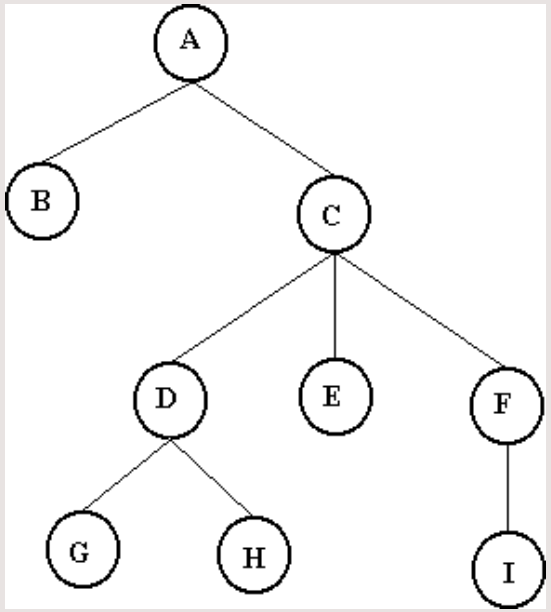
Formas de Representação

Parênteses aninhados: (A (B) (C (D (G) (H)) (E) (F (I))))

Diagrama de Inclusão:



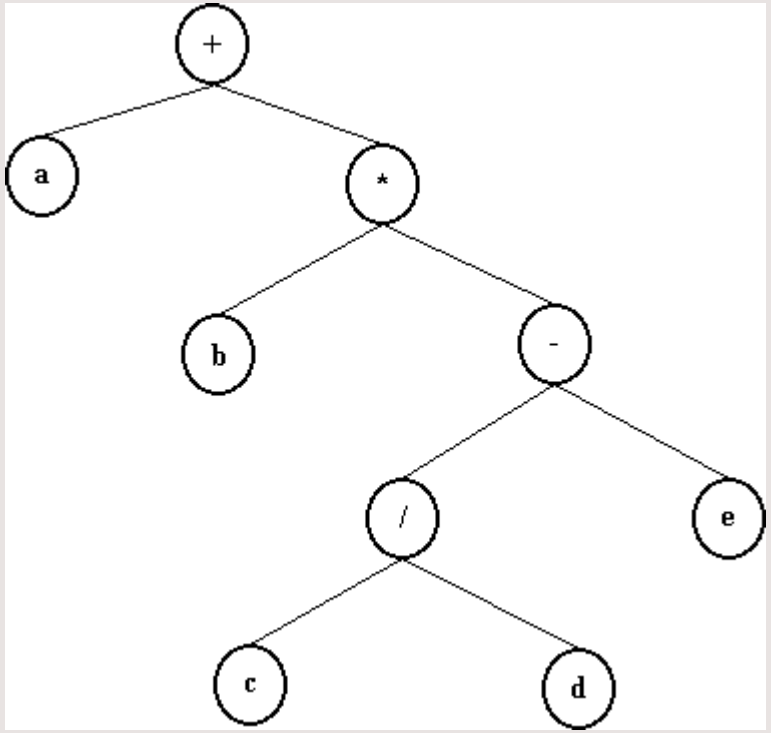
Representação Hierárquica:



Exemplo

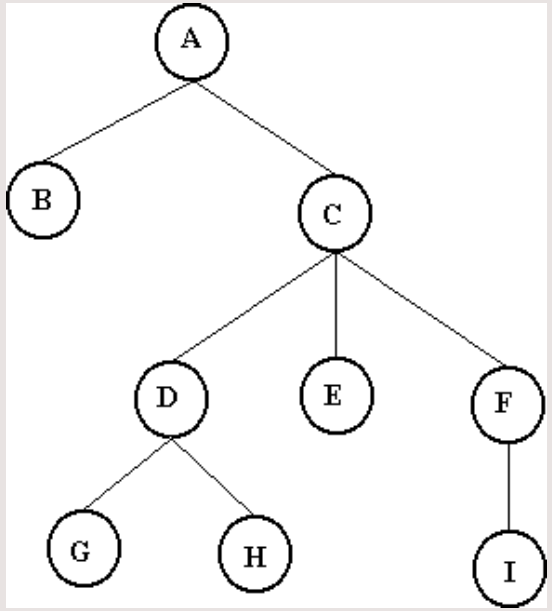
Árvore Sintática:

- Representação da expressão aritmética: (a + (b * (c / d - e)))



Conceitos Básicos

- Seja a árvore exibida na imagem ao lado, **T** = {A, B, ...}



- A árvore **T** possui duas subárvores:
  - T_B e T_C onde T_B = { B } e T_C = {C, D, ...}

- A subárvore T_C possui 3 subárvores:
  - T_D, T_E e T_F onde T_D = {D, G, H}, T_E = {E}, T_F = {F, I}

- As subárvores T_B, T_E, T_G, T_H, T_I possuem apenas o nó raiz e nenhuma subárvore.

Conceitos Básicos

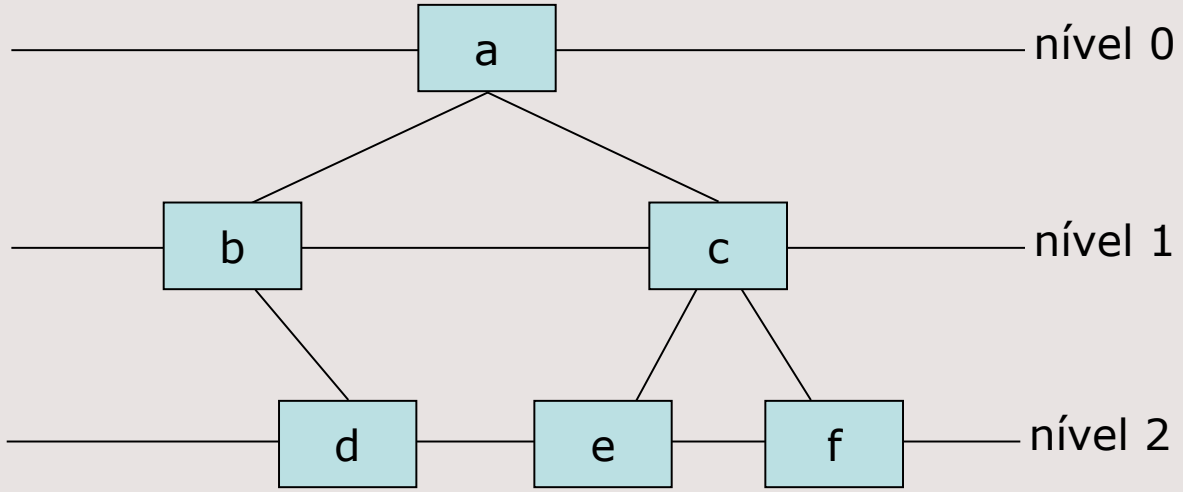
- Nos referimos aos nós como filhos, pais, tios, irmãos etc.
- *Grau de saída*: número de filhos de um nó
- Nó folha (grau de saída igual a zero) e nó interior (grau de saída diferente de zero)
- *Grau de uma árvore*: grau de saída máximo
- *Floresta*: conjunto de árvores (zero ou mais)

Conceitos Básicos

- *Caminho*:
  - É uma sequência de nós distintos v_1, v_2, ..., v_k, tal que existe sempre entre dois nós consecutivos (isto é, entre v_1 e v_2, entre v_2 e v_3, ... , $v_{(k-1)}$ e v_k) a relação "é filho de" ou "é pai de".
- *Comprimento do Caminho*:
  - Um caminho de v_k vértices é obtido pela sequência de $k - 1$ pares. O valor $k - 1$ é o comprimento do caminho.
- *Nível* ou *profundidade de um nó*:
  - É o número de nós do caminho da raiz até o nó.

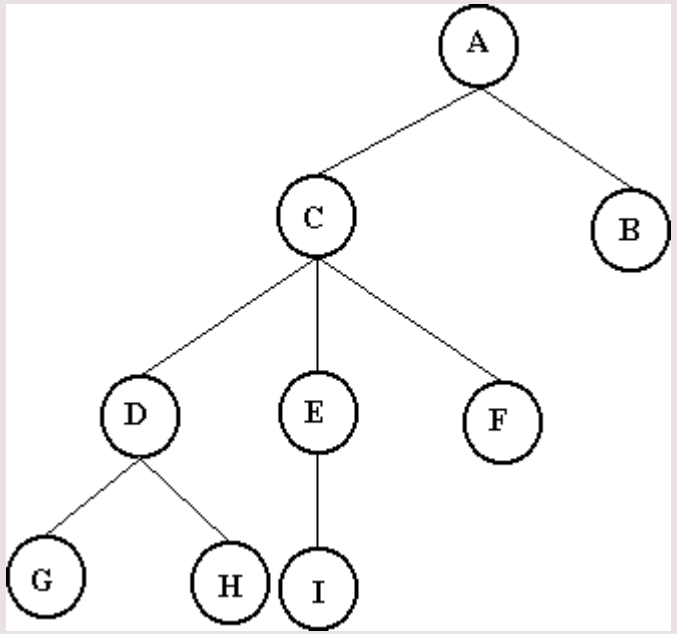
Conceitos Básicos

- Nível de um nó
  - A raiz está no nível 0 (alguns autores adotam 1), seus filhos diretos no nível 1, ...
  - o último nível da árvore é a altura da árvore



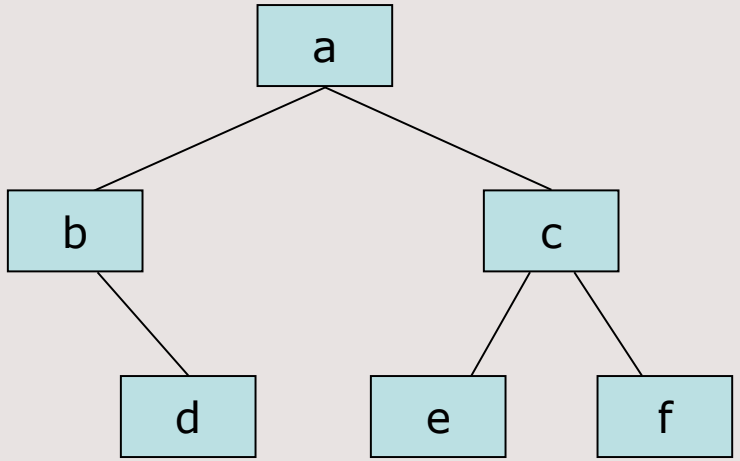
Conceitos Básicos

- *Árvore Ordenada*: é aquela na qual filhos de cada nó estão ordenados. Assume-se ordenação da esquerda para a direita.
  - Esta árvore é ordenada?



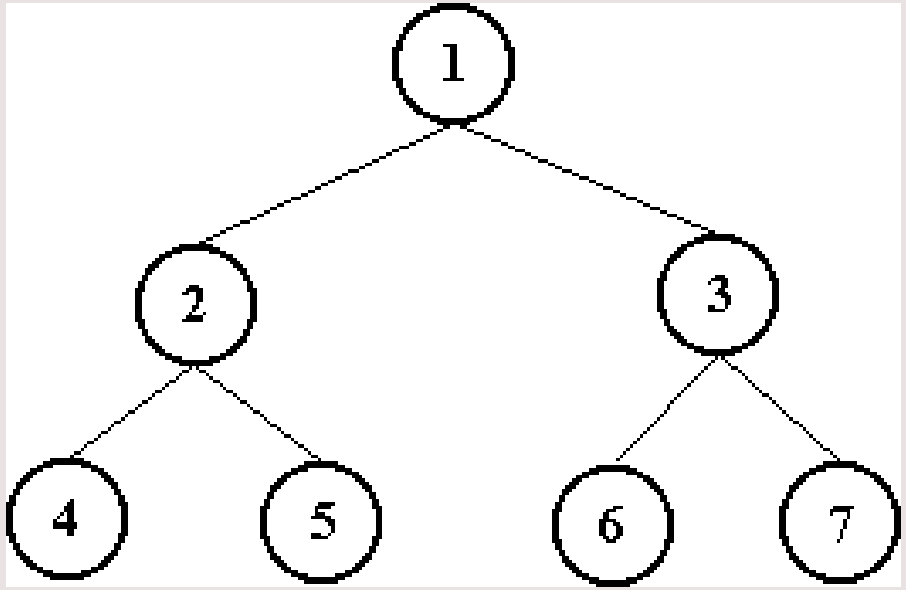
Conceitos Básicos

- Propriedade das árvores:
  - Existe apenas um caminho a partir da raiz para qualquer nó
- *Altura* de uma árvore
  - Por definição, é o comprimento do *caminho mais longo* da raiz até uma das folhas.
  - A altura de uma árvore com um único nó raiz é 0.
  - A altura de uma árvore vazia é –1.
- *O esforço computacional necessário para alcançar qualquer nó da árvore é proporcional à altura da árvore.*
- Exemplo: h = 2



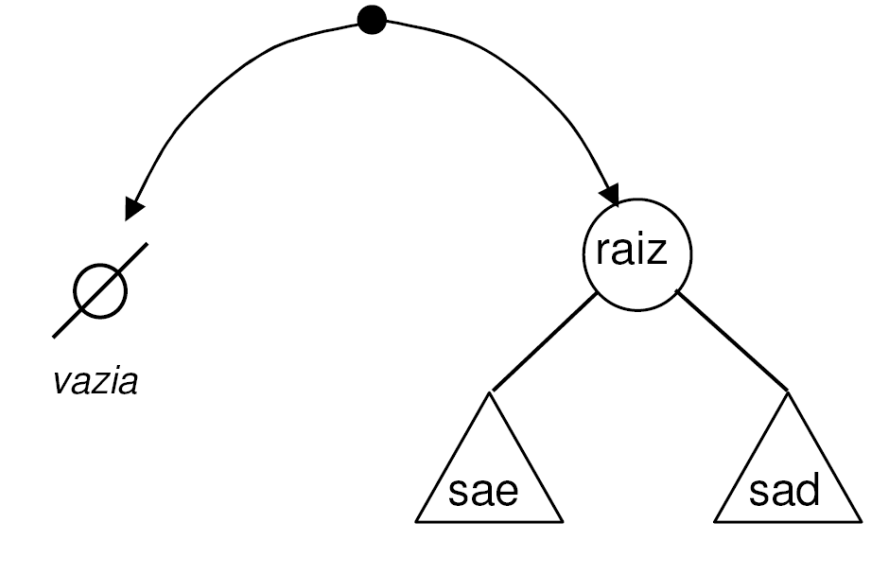
Conceitos Básicos

- *Árvore Cheia*: Uma árvore de grau *d* é uma árvore cheia se possui o número máximo de nós, isto é, todos os nós têm número máximo de filhos exceto as folhas, e todas as folhas estão na mesma altura.
- *Exemplo*: Árvore cheia de grau 2



Árvore Binária

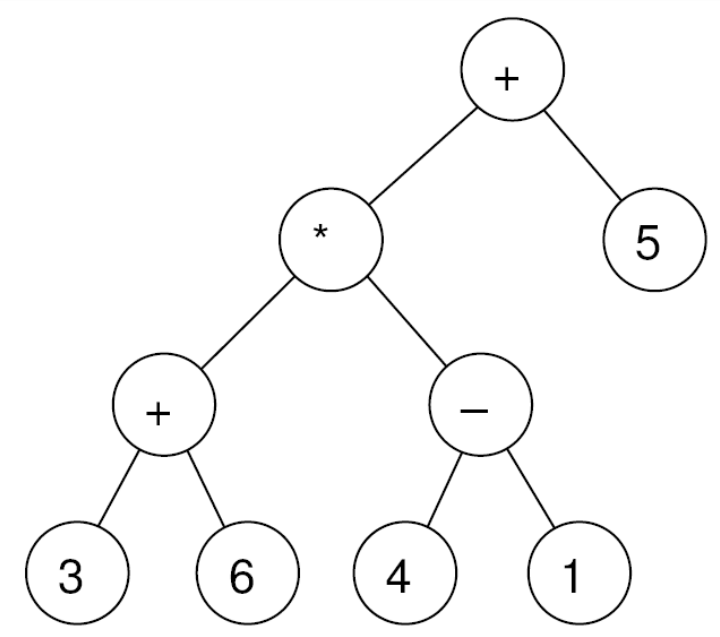
- Uma árvore em que cada nó tem zero, um ou dois filhos
- Uma árvore binária é:
  - uma árvore vazia; ou
  - um nó raiz com duas sub-árvores:
    - a subárvore da direita (sad)
    - a subárvore da esquerda (sae)



Árvore Binária

Exemplo: Árvore binária representando expressões aritméticas de operadores binários (Obs.: existe um '–' unário também, que significa: “mudar sinal”)

- Nós folhas representam os *operandos*
- Nós internos representam os *operadores*
- (3+6)*(4 – 1)+5



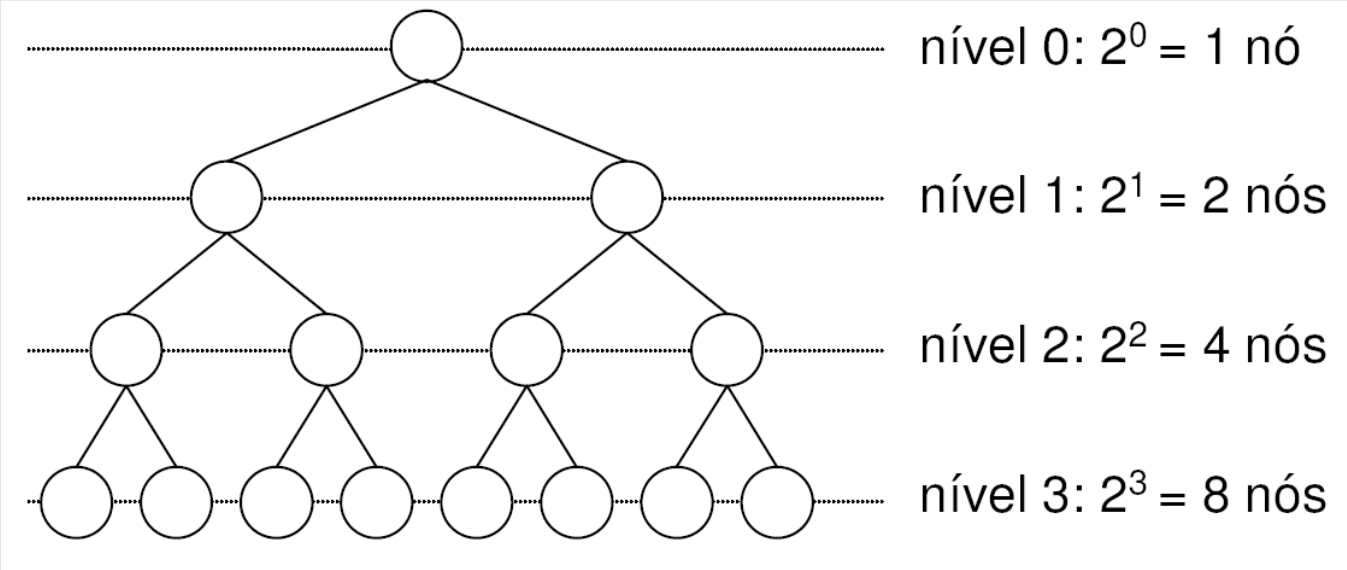
Árvore Binária

- *Árvore Degenerada*
  - Nós internos têm uma única subárvore associada
  - Vira uma estrutura linear
  - Arvore de altura h tem $n = h+1$ nós

- *Altura de uma árvore binária*
  - Importante medida de eficiência (visitação do nó)
  - Árvore com n nós:
    - Altura mínima proporcional a $\log_2 n$ (árvore binária cheia)
    - Altura máxima proporcional a n (árvore degenerada)

Árvore Binária

- *Árvore binária cheia*: todos os seus nós internos têm duas subárvores associadas
- Número de nós *por nível*: $n_n = 2^h$.



Número *total* de nós em uma árvore cheia de altura *h*:

$$n_t = 2^{h+1} - 1$$

No exemplo ao lado:

$$n_t = 2^{3+1} - 1 = 2^4 - 1 = 16 - 1 = 15$$

Árvore Binária

- Representação: ponteiro para o nó raiz
- Representação de um nó na árvore:
  - Estrutura contendo:
    - A informação propriamente dita (exemplo: um caractere, ou inteiro)
    - Dois ponteiros para as subárvores, à esquerda e à direita

```
struct Arv {
    char info;
    struct arv* esq;
    struct arv* dir;
};
```

Árvore Binária

- A implementação das funções de árvore em geral são recursivas.

```
Arv* arv_criavazia (void){
    return NULL;
}


int arv_vazia (Arv* a){
    return a == NULL;
}
```

```
Arv* arv_cria (char c, Arv* sae, Arv* sad){
    Arv* p=(arv*)malloc(sizeof(Arv));
    p->info = c;
    p->esq = sae;
    p->dir = sad;
    return p;
}
```

Árvore Binária

```
Arv* arv_libera (Arv* a){
    if (!arv_vazia(a)){
      arv_libera (a->esq); /* libera sae */
      arv_libera (a->dir); /* libera sad */
      free(a); /* libera raiz */
    }
    return NULL;
}
```

Árvore Binária

```
boolean arv_pertence (Arv* a, char c){
    if (arv_vazia(a))
            return FALSE; /* árvore vazia: não encontrou */
    boolean p = a->info==c;
    if( p ) return p;
    p = arv_pertence (a->esq, c);
    if( p ) return p;
    return arv_pertence (a->dir, c);
}
```

Árvore Binária

```
void arv_imprime (Arv* a){
    if (!arv_vazia(a)){
            printf("%c ", a->info); /* mostra raiz */
            arv_imprime(a->esq); /* mostra sae */
            arv_imprime(a->dir); /* mostra sad */
    }
}
```

Árvore Binária

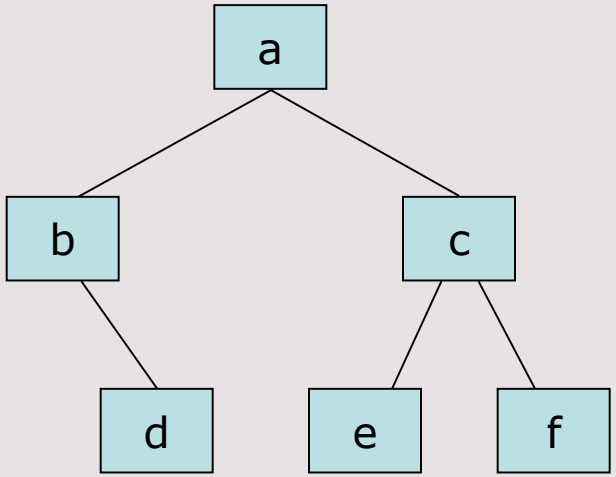
```
boolean arv_pertence (Arv* a, char c){
    if (arv_vazia(a))
        return FALSE; /* árvore vazia: não encontrou */
    boolean p = a->info==c;
    if( p ) return p;
    p = arv_pertence (a->esq,c);
    if( p ) return p;
    return arv_pertence (a->dir,c);
}
```

Árvore Binária

```
Arv* arv_libera (Arv* a){
    if (!arv_vazia(a)){
       arv_libera(a->esq); /* libera sae */
       arv_libera(a->dir); /* libera sad */
       free(a); /* libera raiz */
    }
    return NULL;
}
```

Atravessamento

- Percurso pelos nós da árvore.
- *Pré-ordem*:
  - trata raiz, percorre sae, percorre sad
  - exemplo: a b d c e f
- *Ordem simétrica* (ou *Em-Ordem*):
  - percorre sae, trata raiz, percorre sad
  - exemplo: b d a e c f
- *Pós-ordem*:
  - percorre sae, percorre sad, trata raiz
  - exemplo: d b e f c a



Exercício

- Execute os percursos:
  - Pré-ordem:

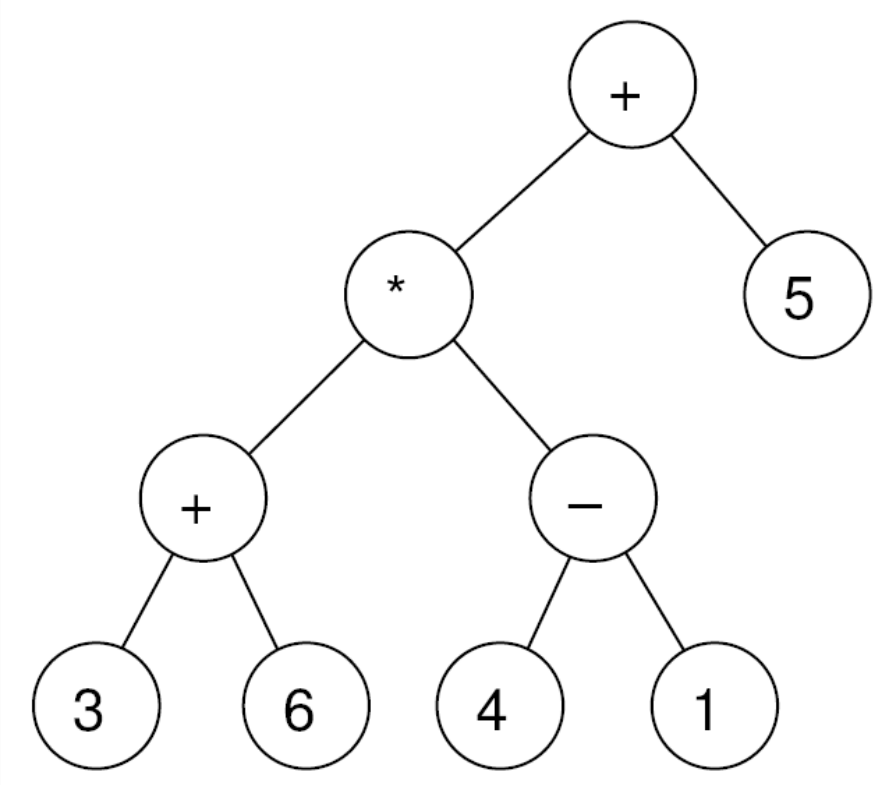
    +*+36-415

  - Em-ordem:

    3+6*4-1+5

  - Pós-ordem

    36+41-*5+



Atravessamento

- Pré-ordem – implementação recursiva

```
void arv_preordem (Arv* a)
{
    if (!arv_vazia(a))
    {
        processa(a); // por exemplo imprime
        arv_preordem(a->esq);
        arv_preordem(a->dir);
    }
}
```

Atravessamento

- Em-ordem – implementação recursiva

```
void arv_inordem (Arv* a)
{
    if (!arv_vazia(a))
    {
        arv_inordem (a->esq);
        processa (a); // por exemplo imprime
        arv_inordem (a->dir);
    }
}
```

Atravessamento

- Pós-ordem – implementação recursiva

```
void arv_posordem (Arv* a)
{
    if (!arv_vazia(a))
    {
        arv_posordem (a->esq);
        arv_posordem (a->dir);
        processa (a); // por exemplo imprime
    }
}
```

Exercício

- Escrever uma função recursiva que calcule a altura de uma árvore binária dada. A altura de uma árvore é igual ao máximo nível de seus nós.

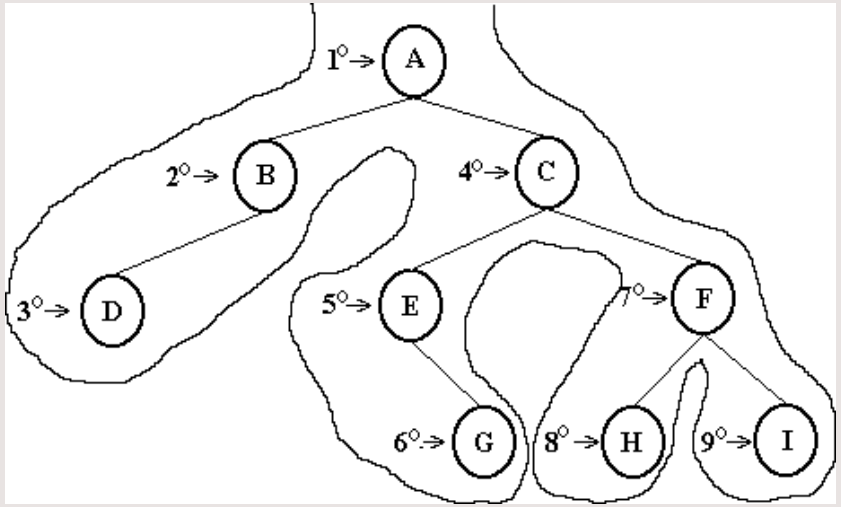
```
#define max2(a,b) {(a > b) ? a : b}

int arv_altura (Arv* a)
{
        if (arv_vazia(a))
                return -1;
        else
                return 1 + max2 (arv_altura (a->esq), arv_altura (a->dir));
}
```

Exercício

- Escrever o algoritmo de visita em Pré-ordem utilizando alocação dinâmica, mas sem utilizar procedimentos recursivos. Utilizar pilha (definindo um vetor que pode ser acessado pelo topo) para saber o endereço da sub-árvore que resta à direita.

  - processar raiz A
  - guardar A na pilha para poder acessar C depois
  - passa à B e processa essa subárvore
  - idem para D
  - retorna B (topo da pilha) para acessar D que é a subárvore esquerda



Exercício

- Resposta:

```
void arv_preordem (Arv* a)
{
        Arv* A[MAX]; //qual seria o valor de max?
        Arv* p; Arv* raiz; int topo;
        boolean acabou;
        topo = 0; p = a; acabou = arv_vazia(a); //inicializações
        while (!acabou) // enquanto houver nós para processar
        {
                while (!arv_vazia(p))
                {
                   processa (p->info);
                   topo++; A[topo] = p;
                   p = p->esq;
                }
                if (topo != 0)
                {
                   p = A[topo]->dir;
                   topo--;
                }
                else {acabou = TRUE;}
        }
}
```

Exercício

- Escrever função para retornar o pai de um dado nó de uma árvore.
  - Dado um item, procura se item existe na árvore (usando algum algoritmo de travessia).
  - Caso positivo retorna o conteúdo do pai do nó .
  - Pode ser recursivo ou não

Exercício

- Sejam ABCDEFGHI e BCAEDGHFI as sequências em pré-ordem e em-ordem de uma mesma árvore binária T.
  - Reconstrua graficamente a árvore T
  - Escreva um algoritmo para criar a representação interna de uma árvore binária a partir de suas sequências em pré-ordem e em-ordem, sendo que cada sequência será fornecida na forma de uma cadeia de caracteres e a árvore criada armazenará um caractere por nodo.
  - Escreva um procedimento para fazer um atravessamento em uma árvore binária por nível

Dúvidas?